



UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería en Informática y Sistemas

Informe final

Proyecto: COMEDOR-SGO

“Sistema de Gestión de Asistencia, Control de Saldos y Optimización de Raciones para el Comedor Universitario de la UNJBG”

Asignatura: *Fundamentos de programación*

Docente: Dr. Hugo Manuel Barraza Vizcarra

D. de Prácticas: Ing. Fernando Eduardo Cohaila Villagra

Integrantes:

Milagros Vanesa Coronado Romero	2026-119034
Joel Joshet Gomez Chachaque	2026-119044
Adomi Sebastian Huanca Mamani	2026-119046

Tacna – Perú
2026



Índice General

I. INTRODUCCIÓN	3
II. PLANTEAMIENTO DEL PROBLEMA	3
III. OBJETIVOS	4
A. OBJETIVO GENERAL	4
B. OBJETIVOS ESPECÍFICOS	4
IV. DESARROLLO DE LA PROPUESTA	5
A. Tecnologías de desarrollo (software utilizado, IDE, lenguajes, extensiones, etc.)	5
B. Descripción de los interesados y usuarios	6
C. Requerimientos funcionales	7
D. Arquitectura de software	8
1. Lista de datos estructurados utilizados (detallar campos y sus significados)	9
2. Lista de funciones clasificadas por áreas.	10
3. Diagramas de flujo y descripción de las funciones más importantes.	11
A. Función cierreHorarioYSaldos	11
B. Función atenderCola	12
C. Función ordenarAlumnosBecados	13
D. Función eliminarAlumnoBecado	14
4. Capturas de menús de opciones más importantes.	15
V. CONCLUSIONES	18



I. INTRODUCCIÓN

El Comedor Universitario de la Universidad Nacional Jorge Basadre Grohmann (UNJBG) cumple un papel fundamental en el bienestar de los estudiantes, al brindar diariamente los servicios de desayuno, almuerzo y cena a los alumnos beneficiarios del programa de becas alimentarias. Este servicio contribuye a mejorar las condiciones de alimentación de los estudiantes y favorece su rendimiento académico.

En este proceso intervienen distintos actores: el personal administrativo del comedor, encargado de la preparación y distribución de las raciones; los estudiantes becados, beneficiarios directos del servicio; y los estudiantes no becados, quienes buscan acceder a las raciones que eventualmente quedan disponibles. La coordinación entre estos actores resulta clave para el correcto funcionamiento del servicio.

Ante las dificultades que se detallan a continuación, se propone desarrollar un sistema de información en C++ que permita gestionar los datos de los estudiantes, registrar la asistencia a los diferentes turnos de alimentación y administrar una cola virtual para la asignación de las raciones disponibles. Con ello, se busca optimizar el aprovechamiento de los alimentos, reducir el desperdicio y mejorar la organización del servicio ofrecido por el comedor universitario.

II. PLANTEAMIENTO DEL PROBLEMA

Actualmente, el comedor universitario no cuenta con un mecanismo que permita registrar oportunamente la asistencia o inasistencia de los estudiantes becados a cada turno de alimentación. Esto impide conocer con anticipación cuántas raciones quedarán sin utilizar, lo que se traduce en desperdicio de alimentos y un uso ineficiente de los recursos destinados al programa de becas alimentarias.



III. OBJETIVOS

A. OBJETIVO GENERAL

- Desarrollar una aplicación en C++ que permita gestionar la información de los estudiantes, controlar la asistencia a los turnos del comedor universitario y administrar una cola virtual para la distribución organizada de las raciones sobrantes, contribuyendo a reducir el desperdicio de alimentos y mejorar la atención a los estudiantes.

B. OBJETIVOS ESPECÍFICOS

- Registrar la información de los estudiantes becados y no becados.
- Listar y buscar estudiantes registrados según distintos criterios (código, nombre, condición de becado).
- Modificar y eliminar registros de estudiantes para mantener la información actualizada.
- Registrar la asistencia de los estudiantes a los turnos de desayuno, almuerzo y cena.
- Calcular la cantidad de raciones disponibles como consecuencia de las inasistencias.
- Implementar una cola virtual para organizar la atención de los estudiantes no becados.
- Generar reportes sobre estudiantes registrados, asistencia, inasistencias, raciones disponibles y atención de la cola virtual.

IV. DESARROLLO DE LA PROPUESTA

A. Tecnologías de desarrollo (software utilizado, IDE, lenguajes, extensiones, etc.)

Para el desarrollo del sistema de gestión del Comedor Universitario se utilizaron las siguientes herramientas y tecnologías:

- **Lenguaje de programación**

C++, empleado para la implementación de la lógica del sistema, el manejo de estructuras de datos y la organización modular del programa mediante funciones.

- **Entorno de desarrollo (IDE)**

Dev-C++, utilizado para la edición, compilación y ejecución del programa, siguiendo la estructura de proyecto recomendada (archivos .dev, .cpp y .h).

- **Control de versiones**

Git, para el control de versiones del código fuente durante el proceso de construcción del proyecto.

GitHub, como repositorio remoto colaborativo, donde se alojó el código fuente y se evidenció la participación de los integrantes del equipo mediante commits.

- **Herramientas de documentación y colaboración**

Google Docs, para la elaboración colaborativa del informe del proyecto.

Google Slides, para el diseño de las diapositivas de exposición.

Google Drive, para el almacenamiento y organización de la documentación, diapositivas y anexos del proyecto.

B. Descripción de los interesados y usuarios

En el desarrollo e implementación del sistema intervienen distintos actores, cada uno con un rol e interés particular en su funcionamiento:

- Personal administrativo del comedor universitario

Es el usuario principal del sistema. Se encarga de registrar a los estudiantes, controlar la asistencia a cada turno de alimentación, verificar la cantidad de raciones disponibles y gestionar la cola virtual para la atención de los estudiantes no becados. Su interés principal es contar con una herramienta que le permita organizar el servicio de manera ágil y reducir el desperdicio de alimentos.

- Estudiantes becados

Son los beneficiarios directos del programa de alimentación. Su relación con el sistema es indirecta, ya que su asistencia o inasistencia a los turnos es registrada por el personal administrativo. Su interés radica en recibir un servicio ordenado y eficiente durante los horarios de atención.

- Estudiantes no becados

Son usuarios interesados en acceder a las raciones sobrantes que resultan de las inasistencias. Su interés principal es conocer con mayor certeza la disponibilidad de raciones y ser atendidos de manera ordenada mediante la cola virtual, evitando así la incertidumbre y las largas esperas actuales.

C. Requerimientos funcionales

Los requerimientos funcionales describen las funciones que el sistema COMEDOR-SGO debe cumplir para satisfacer las necesidades del Comedor Universitario de la Universidad Nacional Jorge Basadre Grohmann. El sistema permitirá administrar la asistencia de alumnos becados, controlar las raciones disponibles y gestionar la distribución de alimentos sobrantes mediante una cola virtual para estudiantes no becados.

Las principales funciones son las siguientes:

RF-01. Gestión de alumnos becados

El sistema debe permitir registrar, consultar, modificar, eliminar y ordenar la información de los alumnos becados autorizados para utilizar el comedor universitario.

RF-02. Registro de asistencia

El sistema debe permitir registrar la asistencia de un alumno becado mediante su código universitario para cualquiera de los tres turnos de atención: desayuno, almuerzo y cena.

RF-03. Consulta de alumnos asistentes

El sistema debe mostrar la lista de alumnos becados que registraron asistencia durante un turno determinado.

RF-04. Consulta de alumnos faltantes

El sistema debe identificar y mostrar los alumnos becados que no registraron asistencia en el turno correspondiente, permitiendo estimar la cantidad de raciones que podrían quedar disponibles.

RF-05. Registro de alumnos no becados

El sistema debe permitir registrar alumnos no becados en una cola virtual de espera, almacenando su código universitario, nombre y turno solicitado.



RF-06. Administración de la cola virtual

El sistema debe mostrar la lista de alumnos registrados en la cola virtual y permitir su ordenamiento para mantener una atención organizada.

RF-07. Configuración de raciones diarias

El sistema debe permitir registrar la cantidad de raciones disponibles para cada turno del día (desayuno, almuerzo y cena), información necesaria para calcular el saldo de alimentos.

RF-08. Cálculo de raciones sobrantes

El sistema debe calcular automáticamente las raciones sobrantes considerando la cantidad programada de alimentos y el número de alumnos becados que asistieron al comedor.

RF-09. Atención automática de la cola virtual

El sistema debe asignar las raciones sobrantes a los alumnos registrados en la cola virtual respetando el orden establecido hasta agotar la disponibilidad.

RF-10. Generación de reportes

El sistema debe generar reportes relacionados con la asistencia, alumnos faltantes, estado de la cola virtual y cantidad de raciones sobrantes por cada turno.

D. Arquitectura de software

El sistema COMEDOR-SGO fue desarrollado utilizando una arquitectura modular basada en programación estructurada en lenguaje C++. Cada módulo del sistema se encuentra organizado mediante archivos de cabecera (.h) y archivos de implementación (.cpp), permitiendo distribuir el desarrollo entre los integrantes del equipo y facilitando el mantenimiento del software.

La aplicación se ejecuta desde el archivo **main.cpp**, el cual controla el flujo principal del programa y coordina la interacción entre los diferentes módulos.

Los módulos implementados son:

- **main.cpp**: controla la ejecución general del sistema.
- **menu.cpp / menu.h**: administra los menús y la navegación entre opciones.
- **estructuras.cpp / estructuras.h**: contiene las estructuras de datos utilizadas en el sistema.
- **alumnos.cpp / alumnos.h**: administra la información y asistencia de los alumnos becados.
- **colavirtual.cpp / colavirtual.h**: gestiona la cola virtual de alumnos no becados.
- **reportes.cpp / reportes.h**: calcula las raciones sobrantes y genera los reportes correspondientes.

Esta organización modular permite separar responsabilidades, facilitar el trabajo colaborativo y mejorar la reutilización del código.

1. Lista de datos estructurados utilizados (detallar campos y sus significados)

Estructura: AlumnoBecado

Campo	Tipo	Significado
codigo	string	Código universitario del alumno.
nombre	string	Nombre completo del estudiante.
asistio[3]	bool	Arreglo que almacena la asistencia del alumno para desayuno, almuerzo y cena.

Estructura: AlumnoNoBecado

Campo	Tipo	Significado
codigo	string	Código universitario del estudiante.

nombre	string	Nombre completo del estudiante.
turnoSolicitado	int	Turno para el cual solicita una ración (desayuno, almuerzo o cena).
atendido	bool	Indica si el estudiante ya recibió una ración disponible.

Constantes utilizadas

Constante	Significado
TOTAL_TURNOS	Número total de turnos del comedor (3).
CAPACIDAD_ALUMNOS	Cantidad máxima de alumnos becados registrados.
CAPACIDAD_COLA	Cantidad máxima de alumnos en la cola virtual.

2. Lista de funciones clasificadas por áreas.

Gestión de alumnos becados

- registrarAlumnoBecado()
- listarAlumnosBecados()
- buscarAlumnoBecado()
- modificarAlumnoBecado()
- eliminarAlumnoBecado()
- ordenarAlumnosBecados()

Estas funciones permiten administrar completamente la información de los estudiantes becados registrados en el sistema.

Control de asistencia

- registrarAsistencia()
- listarAsistentes()
- listarFaltantes()

Estas funciones administran el registro de asistencia para cada turno y permiten conocer qué alumnos asistieron y cuáles no.

Gestión de cola virtual

- registrarAlumnoNoBecado()
- registrarCola()
- mostrarCola()
- ordenarCola()

Estas funciones administran el registro, visualización y organización de los alumnos no becados que esperan acceder a una ración disponible.

Reportes y procesamiento

- cierreHorarioYSaldos()
- atenderCola()

Estas funciones calculan el número de raciones sobrantes y distribuyen automáticamente los alimentos disponibles entre los alumnos registrados en la cola virtual.

Interfaz de usuario

- mostrarMenuPrincipal()
- mostrarSubmenuAlumnos()
- mostrarSubmenuGestion()

Estas funciones presentan los menús del sistema y permiten la navegación entre las distintas opciones disponibles.

Estructuras de datos

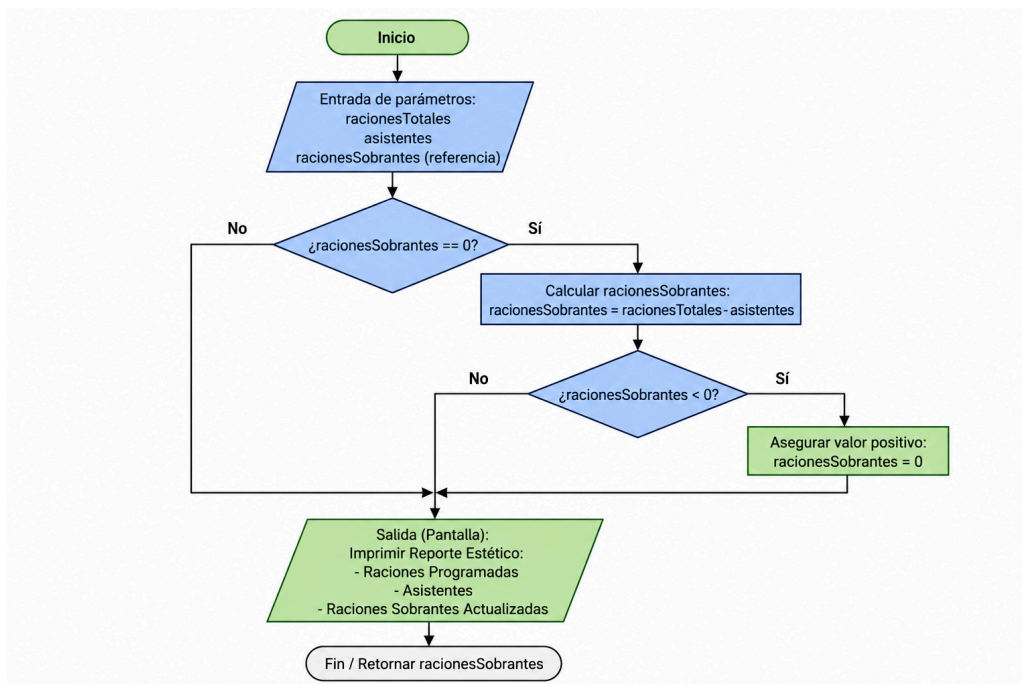
El sistema utiliza estructuras (struct) para representar la información de alumnos becados y no becados, permitiendo almacenar y manipular los datos de forma organizada durante la ejecución del programa.

3. Diagramas de flujo y descripción de las funciones más importantes.

A continuación, se describen detalladamente los procesos lógicos de las dos funciones de procesamiento más críticas del sistema, adjuntando sus respectivos diagramas de flujo generados.

A. Función cierreHorarioYSaldos

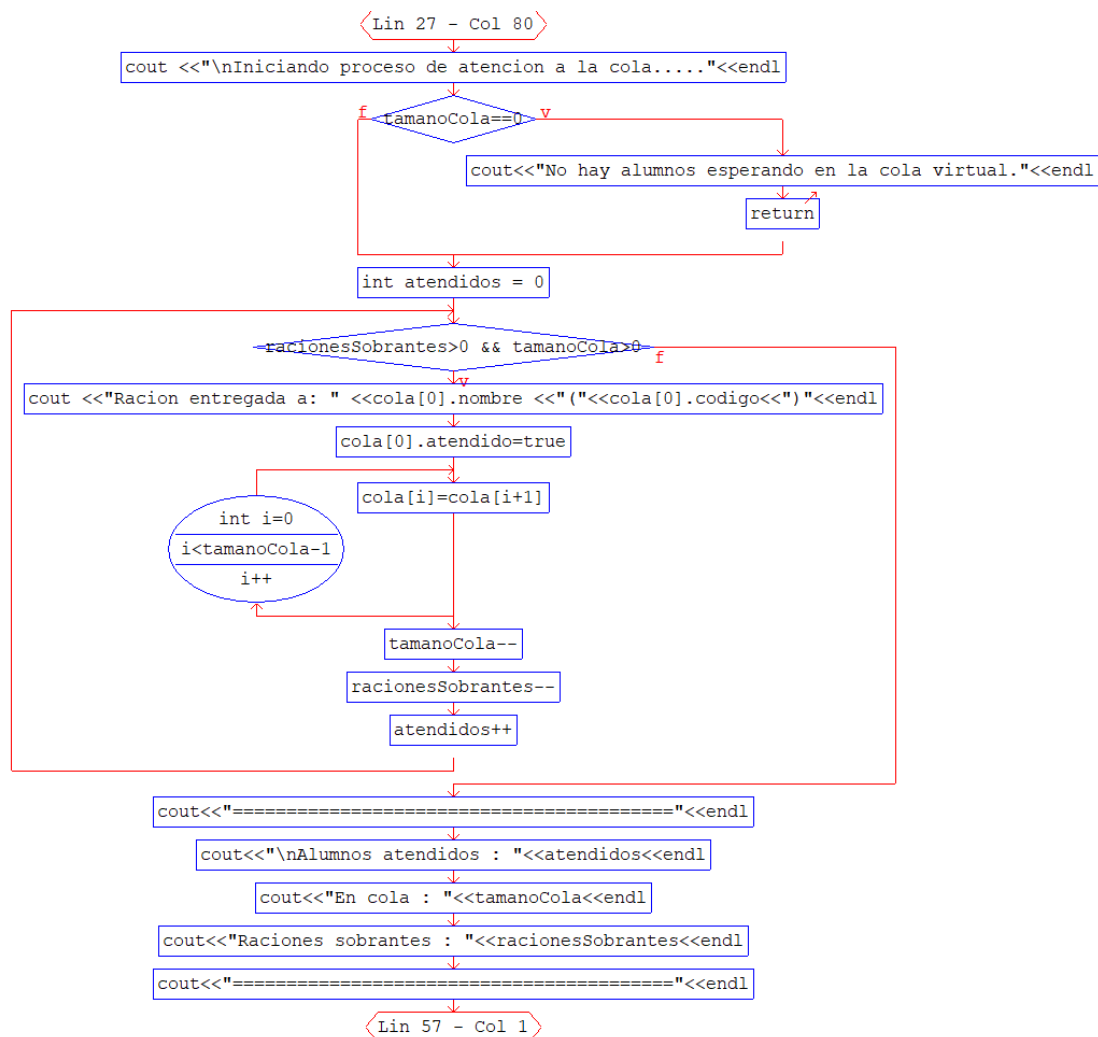
Descripción técnica: Esta función procesa el balance final del turno actual. Evalúa la cantidad de raciones programadas en contraste con los asistentes reales registrados. Cuenta con un condicional de control fundamental: la operación matemática de resta solo se ejecuta si la variable `racionesSobrantes` es igual a cero (es decir, la primera vez que se



consulta la opción en el turno). Esto protege el stock de raciones sobrantes, evitando que se reinicie o se altere si el usuario vuelve a consultar esta opción después de haber despachado platos a la cola virtual.

B. Función atenderCola

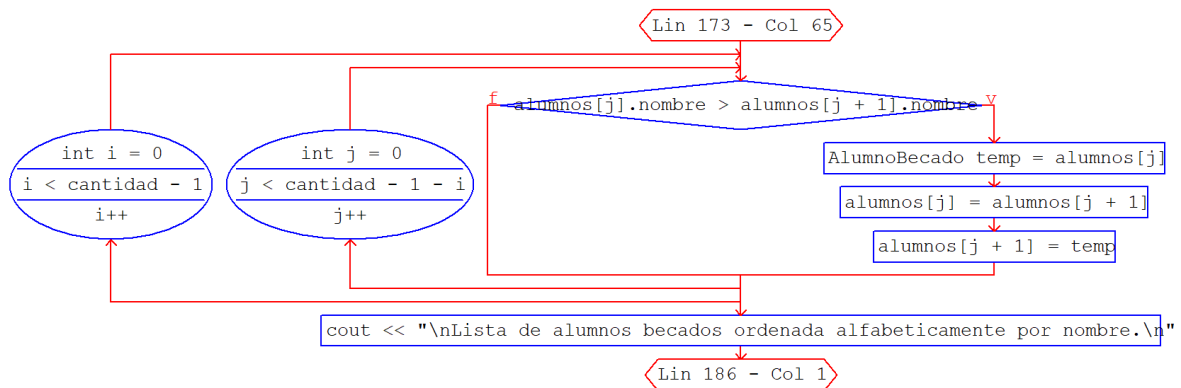
- Descripción técnica:** Modela el despacho físico y la entrega de platos a los estudiantes no becados que se encuentran en lista de espera utilizando una lógica de cola (FIFO). La función valida que existan platos disponibles y que la cola no esté vacía. En cada iteración, entrega la ración al primer alumno de la fila (`cola[0]`), desplaza de forma secuencial a los siguientes estudiantes hacia adelante para simular el avance de la cola, y decrementa tanto las raciones disponibles como el tamaño de la cola mediante **paso por referencia (&)**, lo que garantiza que el descuento impacte de manera real y directa en el archivo `main.cpp`.



C. Función ordenarAlumnosBecados

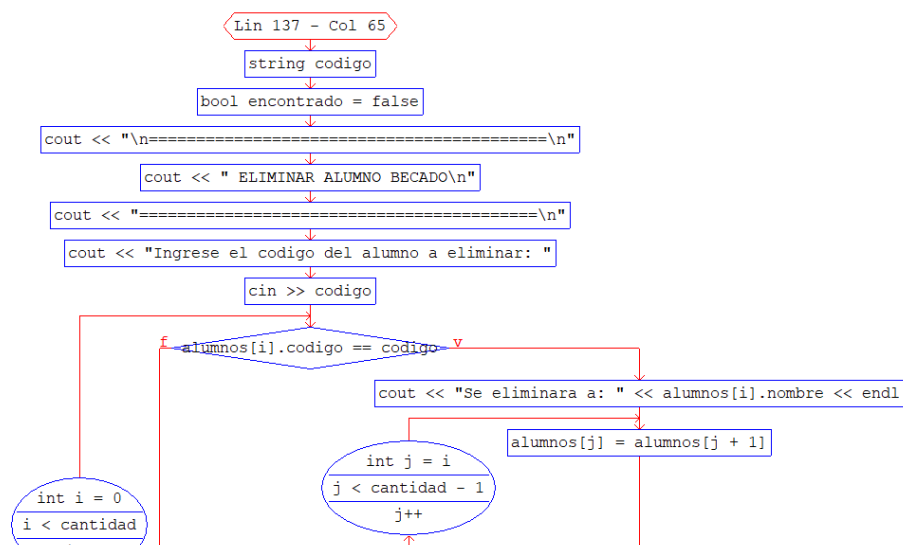
- Descripción técnica:** Esta función implementa un algoritmo de ordenamiento de orden lineal-cuadrático (Método Burbuja o *Bubble Sort*) para organizar

alfabéticamente el arreglo de estructuras de alumnos becados en base a su miembro `.nombre`. Utiliza dos bucles anidados (`for`) para comparar pares de elementos adyacentes e intercambiarlos de posición mediante una variable auxiliar de tipo `AlumnoBecado` cuando no cumplen con el criterio de ordenamiento, permitiendo que las búsquedas y visualizaciones posteriores se realicen de forma estructurada.



D. Función `eliminarA_lumnoBecado`

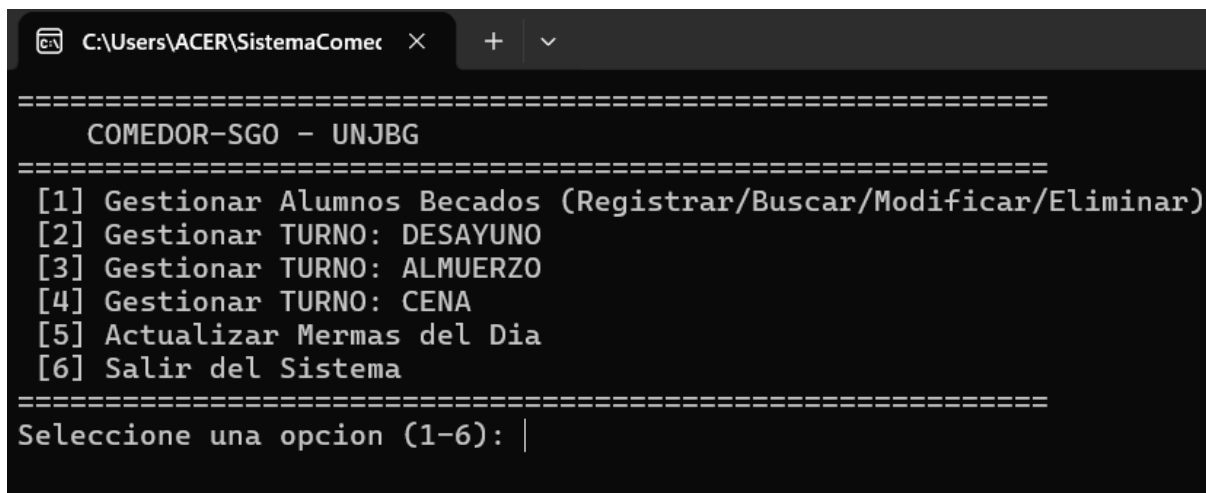
- Descripción técnica:** Modela la baja física de un alumno del sistema. Recibe la variable `cantidad` de manera interactiva por medio de **paso por referencia (&)**. Tras realizar una búsqueda lineal del código universitario solicitado, si este es hallado, la función ejecuta un desplazamiento de elementos hacia la izquierda mediante un bucle `for` (sobreescribiendo el registro eliminado con el dato del elemento adyacente posterior). Finalmente, reduce en una unidad el contador físico de alumnos en memoria (`cantidad--`).



4. Capturas de menús de opciones más importantes.

Parte 1: El Menú Principal (Turno Desayuno, Almuerzo o Cena)

- **Figura 1:** *Interfaz del Menú Principal del Sistema.* > Se muestra la consola de comandos con el menú de opciones estructurado para el control del turno activo. Se evidencia el diseño modular dividido en gestion de alumnos becados (registrar/modificar/buscar/eliminar), control de los 3 turnos de



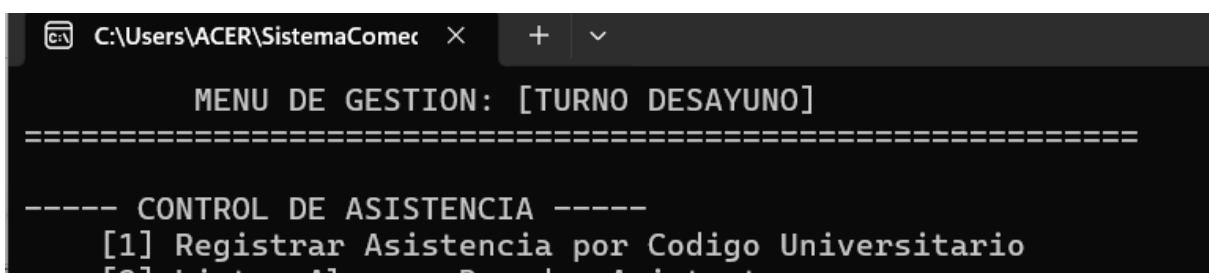
```
=====
COMEDOR-SGO - UNJBG
=====
[1] Gestionar Alumnos Becados (Registrar/Buscar/Modificar/Eliminar)
[2] Gestionar TURNO: DESAYUNO
[3] Gestionar TURNO: ALMUERZO
[4] Gestionar TURNO: CENA
[5] Actualizar Mermas del Dia
[6] Salir del Sistema
=====
Seleccione una opcion (1-6): |
```

alimentación e importante, la actualización de mermas por día, además de claro, la opción de cerrar el programa.

A continuación, actualizaré las mermas a 10 por día como método de prueba:

Parte 2: Gestion de alumnos becados

- **Elección:** Elegimos la opción de guardar alumnos becados para la atención de las raciones. Inmediatamente después registramos a los alumnos no becados en el menú principal usando las opciones 2-4 para ver el uso de la cola virtual, al ser parecido el proceso, elegiremos el desayuno



```
=====
MENU DE GESTION: [TURNO DESAYUNO]
=====
----- CONTROL DE ASISTENCIA -----
[1] Registrar Asistencia por Codigo Universitario
[2] Registrar Asistencia por Documento de Identidad
[3] Registrar Asistencia por Documento de Identidad
[4] Registrar Asistencia por Documento de Identidad
[5] Registrar Asistencia por Documento de Identidad
[6] Registrar Asistencia por Documento de Identidad
=====
Seleccione una opcion (1-6): |
```



Con la opción de listar alumnos faltantes, podremos saber quienes son los becados que no vinieron a recoger su comida. Como este proceso agiliza a los becados no realizan cola, mientras que los no becados si, por ende los alumnos que ingresé como no becados pueden verse con la opción 5.

```
=====
Seleccione una opcion (1-9): 3
=====
                ALUMNOS BECADOS FALTANTES
=====
CODIGO          NOMBRE
-----
2026-119046      Adomi Sebastian Huanca Mamani
2026-119047      Giovanni Imannol Mamani Mamani
Presione una tecla para continuar . . . |
```

```
=====
Seleccione una opcion (1-9): 5
=====
| ESTADO DE LA COLA VIRTUAL |
=====
1.Codigo: 2024-118034 | Nombre: Marco Romero Choque | Turno: 0 | Estado: En espera
2. Codigo: 2022-14502 | Nombre: Gian Cotrado Laura | Turno: 0 | Estado: En espera
=====
Presione una tecla para continuar . . . |
```

Parte 3: Reporte de Saldos Sincronizado.

- **Despacho de alumnos:** con el menú base, registramos la asistencia de los becados usando la opción 1, así poniendo si ingresó o no para que se descuenta de las raciones diarias.

```
=====
Seleccione una opcion (1-9): 1
=====
                REGISTRO DE ASISTENCIA
=====
Ingrese el codigo universitario: 2026-119046

Asistencia registrada correctamente.
Bienvenido(a): Adomi Sebastian Huanca Mamani
Presione una tecla para continuar . . . |
```


Ahora con los no becados solo basta con realizar la opción 8 de atender la cola virtual, realizándose la siguiente operación:

```
=====
Seleccione una opcion (1-9): 8

Iniciando proceso de atencion a la cola.....
Racion entregada a: Marco Romero Choque(2024-118034)
Racion entregada a: Gian Cotrado Laura(2022-14502)
=====

Alumnos atendidos : 2
En cola      : 0
Raciones sobrantes : 7
=====
Presione una tecla para continuar . . . |
```

Ahora usando la opción 7 miramos el cierre de horario para verificar que todo se unió perfectamente (los alumnos que asistieron solo cuentan a becados):

```
=====
Seleccione una opcion (1-9): 7
=====
|      CIERRE DE HORARIO Y SALDOS      |
=====
Raciones programadas totales: 10
Alumnos que ya asistieron   : 1
-----
Raciones Sobrantes : 7
=====
Presione una tecla para continuar . . . |
```

V. CONCLUSIONES

- **Primera (modularidad y flujo de control):** La representación del sistema mediante diagramas de flujo estructurados (como los diseñados para los procesos de eliminación, ordenamiento y cálculo de saldos) permitió identificar y depurar de forma preventiva cuellos de botella en la lógica de control. Esto garantizó que bucles complejos y condicionales anidados se ejecuten de manera óptima antes de pasar a la fase de compilación en Dev-C++.
- **Segunda (persistencia y paso por referencia):** El uso de **paso por referencia (&)** en variables críticas del sistema (como la cantidad en la



eliminación de alumnos y las raciones Sobrantes en la cola virtual) demostró ser indispensable en C++. Este método asegura que cualquier modificación realizada dentro de las funciones se sincronice en tiempo real y con total consistencia en el menú principal (`main.cpp`), evitando la duplicidad innecesaria de datos en memoria.

- **Tercera (optimización de estructuras y algoritmos):** La implementación de algoritmos clásicos de las ciencias de la computación —como el método burbuja en ordenar `AlumnosBecados` y el desplazamiento dinámico de memoria hacia la izquierda al momento de eliminar `AlumnoBecado`— dota al sistema de un comportamiento realista y robusto. Esto resuelve con precisión matemática las necesidades operativas de la distribución diaria de raciones en el comedor de la **UNJBG**.

Gracias a esto, podemos concluir que el problema inicial fue solucionado con éxito.

